Context engineering

funnamer | 今天修改

一、什么是上下文工程

（1）定义

在传统的提示词工程中，上下文 C 只是一个单一的字符串（即 C=prompt）。而上下文工程将 C 重新概念化为由多个信息组件（$c1,c2,...,cn$）动态组合而成的结构：

$$C=A(c1,c2,...,cn)$$

这里的 A 是一个“组装函数”，负责调度这些由不同函数获取、过滤和格式化的组件。这些组件主要包括六个核心领域：

- *cinstr* **(指令)**：系统指令和规则。
- *cknow* **(知识)**：通过RAG（检索增强生成）或知识图谱获取的外部知识。
- *ctools* **(工具)**：可用外部工具的定义和签名（用于函数调用）。
- *cmem* **(记忆)**：历史交互中保留的持久化信息。
- *cstate* **(状态)**：用户、现实世界或多智能体系统的动态状态。
- *cquery* **(查询)**：用户当下的直接请求。

上下文工程本质上是一个**寻找最优上下文生成函数集合**的数学优化问题。我们的目标是找到一组理想的函数 F（包括检索、选择、组装等），以最大化模型输出的预期质量（Reward），同时受限于模型的最大上下文长度限制（$|C|\leq Lmax$）：

一句话总结就是

> 构建有效的context结构，在上下文中找到 **最小的一组高信息密度 token**，同时最大化模型产生目标结果的概率。

（2）上下文工程与提示词工程有什么不同?

核心焦点的不同：从“如何表达”到“如何统筹”

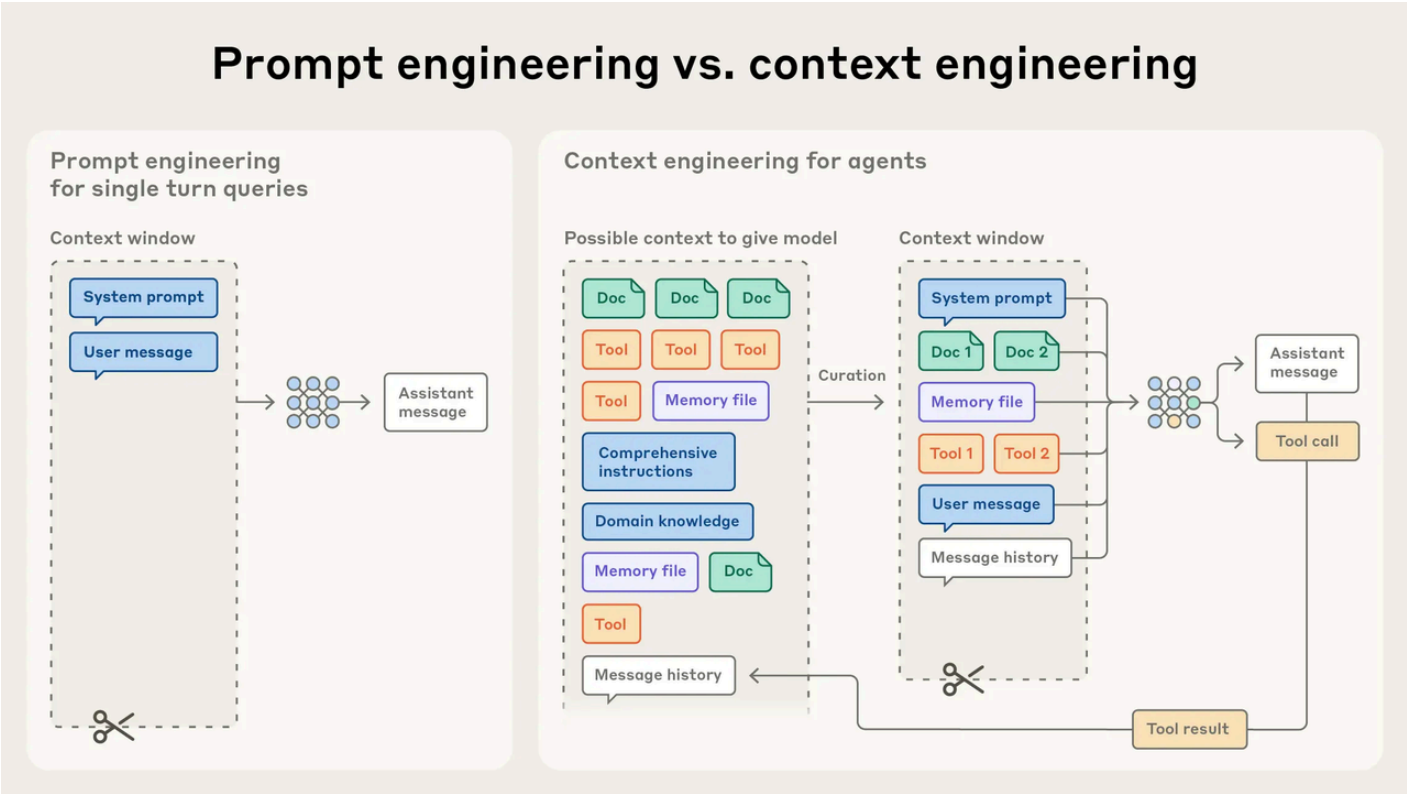
提示词工程 (Prompt Engineering)： 它的核心在于“写（Writing）”和“组织（Organizing）”。主要关注如何遣词造句、如何设计 System Prompt、如何清晰地向大模型下达指令。这在早期的 AI 应用（如单轮对话、文本分类、内容生成）中是绝对的核心。

上下文工程 (Context Engineering)： 它的核心在于“策展（Curating）”和“维护（Maintaining）”。它不仅包含提示词，还包揽了在模型推理期间进入上下文窗口的所有信息。

管理维度的不同：从“静态单点”到“动态全局”

提示词工程 (Prompt Engineering)：视角相对静态。通常是面向“单次（One-shot）”任务进行优化，打磨一段完美的静态文本。

提示词工程 (Prompt Engineering)：视角是全局且动态的。它管理的是整个上下文状态 (Context State)，其中包括：系统指令、工具定义、模型上下文协议 (MCP)、外部数据、以及不断增长的历史消息。



维度	提示词工程 (Prompt Engineering)	上下文工程 (Context Engineering)
核心动作	编写 (Writing)、指令设计	策展 (Curating)、状态维护、信息过滤
主要处理对象	System Prompt、用户输入	提示词 + 工具 + 记忆 + 外部数据 + 历史流
适用场景	单轮对话、单次文本生成/分类	多轮推理、长周期 Agent 任务、复杂状态机
面临的挑战	表达不清晰、模型无法理解指令	上下文污染、Token 超载、注意力预算耗尽
本质定位	沟通技巧（如何把话说明白）	资源调度与信息物流系统（运送什么信息最有价值）

随着智能体的发展，prompt engineering这一范式正在逐步转变为context engineering

当一个 Agent 在循环（Loop）中运行，执行多轮推理和长周期任务时，它会不断生成新的数据（比如工具调用结果、中间思考过程）。这些新数据必须被周期性地提炼和精简 (Cyclically refined)，否则很快就会撑爆模型的注意力预算（Attention Budget）。提示词工程解决不了“历史消息越来越长”或者“外部检索数据太多”的问题，而这正是上下文工程要解决的命题。

上下文工程不再是“如何写好一句话来引导AI”，而是构建一套严密的、模块化的系统架构。它通过科学地调度指令、知识、工具、记忆和状态，让AI系统能够像人类一样，在复杂、模糊、动态且多模态的现实世界中，做出准确的响应。

二、为什么需要上下文工程

（1）实际场景的问题

正如上面所提到的，现实生活中有相当一部分问题需要模型结合实时信息进行回答，需要提供额外信息（这里可以是RAG、search工具等等）以供模型在新的知识上进行推理、回答，但是大量的信息会导致模型的注意力漂移，如何有效的管理上下文，如何组织最少的上下文来让模型完成目标任务十分重要，因此需要context-engineering。

（2）过长上下文带来的问题

大模型并不是全能的，它们在处理信息时面临着严苛的架构瓶颈。上下文工程是用来管理这些瓶颈的必要手段：

- 自注意力机制的计算爆炸：LLM 基于 Transformer 架构，其核心的自注意力机制需要让每一个 Token 与其他所有 Token 建立联系。这会导致 n^2 的计算复杂度和内存开销。随着上下文变长，计算资源会呈二次方级消耗。
- 上下文腐败 (Context Rot) 与注意力预算：Anthropic 明确指出，LLM 就像人类的短期记忆一样，拥有一个有限的“注意力预算（Attention Budget）”。在“大海捞针”测试中发现，随着 Token 数量的增加，模型准确回忆信息的能力会下降（即上下文腐败）。每一个新加入的 Token 都在消耗这个预算，导致注意力被“摊薄”。
- 训练数据的长短分布偏差：在模型的训练数据集中，短序列远比长序列常见。这意味着模型对于“全局/长程依赖”的经验较少，缺乏专门处理超长上下文的参数。虽然有位置编码插值等技术来强行扩展窗口，但这往往会牺牲模型对 Token 位置的精确理解，导致在长上下文中信息检索和逻辑推理能力衰退。

（3）实际业务场景的问题

在商业和实际部署中，粗放的提示词和冗长的上下文会带来致命问题，必须通过工程化手段进行治理：

- 缓解幻觉与不可靠性：如果上下文缺乏精心设计，LLM 极易出现幻觉（Hallucinations）、对输入上下文不忠实、对微小变化过度敏感，或者给出“语法正确但缺乏语义深度”的废话。
- 降低延迟与 Token 成本：在商业应用中，每一次对话都在消耗 API 成本（按 Token 计费）。冗余的上下文不仅拖慢了生成速度（Latency），还直接增加了财务成本。上下文工程通过智能内容过滤和动态上下文优化，在不牺牲回答质量的前提下大幅降低 Token 消耗。

（4）多跳、复杂推理的问题

上下文工程不仅是为了“避坑”，更是为了“拔高”，它能让通用的 LLM 在垂直领域表现出专家级的能力：

- 结构化推理与思维链：ToT（思维树）和规划（Planning）等结构化提示技术，上下文工程引导模型拆解复杂问题，进行多步中间推理。
- 无需微调的灵活适配 (In-Context Learning)：上下文工程允许模型通过零样本（Zero-shot）、少样本（Few-shot）演示和角色设定，直接适应新任务（如代码摘要、Bug 修复），而无需进行昂贵的底层权重微调。并且精心设计的示例能让代码修复等特定任务的准确率飙升。
- 专业领域的性能飞跃：在硬件设计、代码调试等高度专业化的领域，通过 RAG（检索增强生成）和领域特定的上下文框架，可以直接将精确的外部知识注入模型，实现高达数倍的性能提升。

（5）核心理念：将上下文视为“稀缺资源”

传统的提示词工程往往把上下文当成一个“垃圾桶”，把所有可能相关的信息都塞进去。而上下文工程的核心哲学发生了转变：

- **信息密度最大化**：因为上下文是具有“边际收益递减”特性的稀缺资源，所以必须对其进行严格的策展（Curate）。上下文工程利用压缩、选择和过滤机制，只保留信息密度最高的内容。
- **动态与自适应**：未来的上下文工程将更加动态化，比如根据当前任务的需要，精准地在代码的语法、语义、执行流和文档之间进行上下文类型的切换与调配。

三、如何设计上下文中的每一个组件?

（1）System prompt

设计原则：

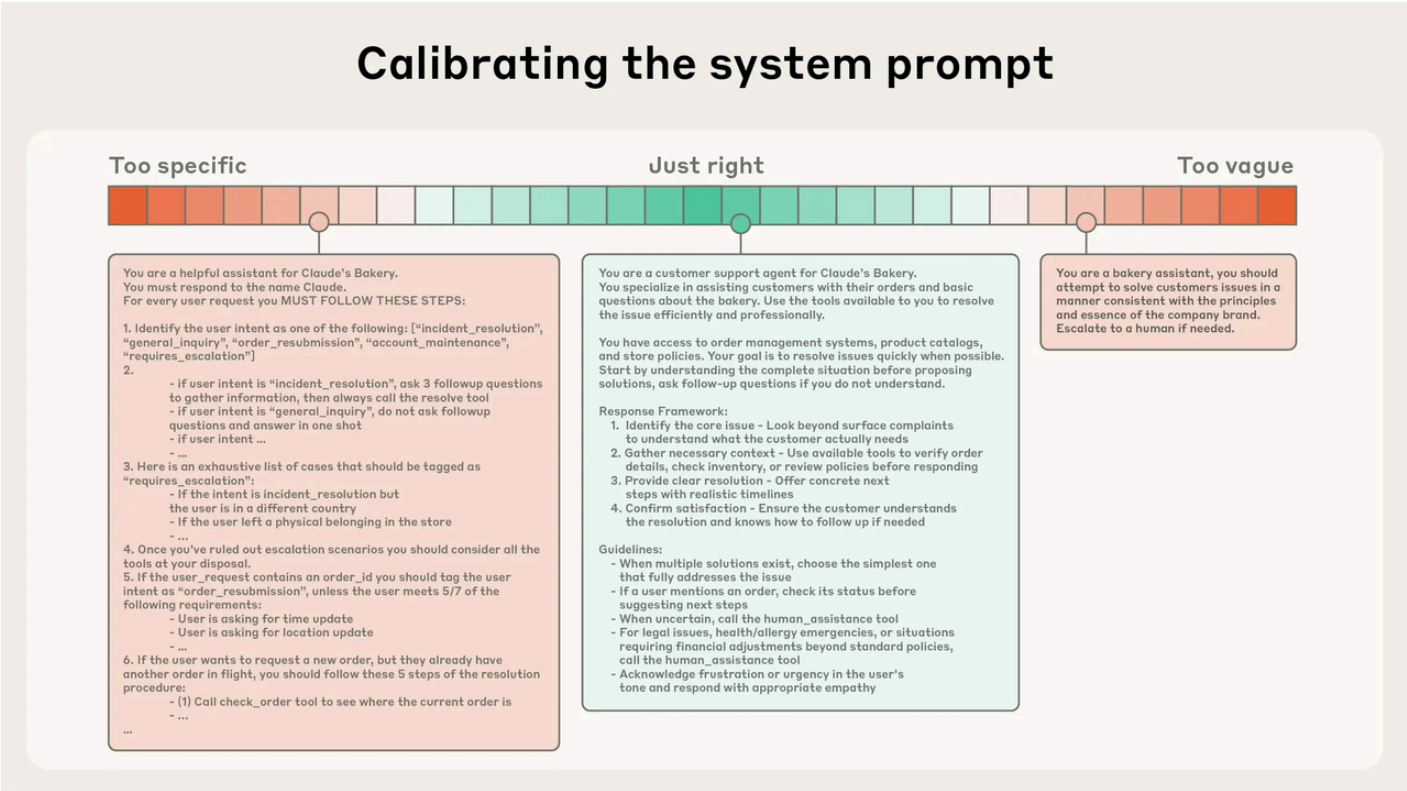
- 启发式指导，让模型思考、推理

- 指导具体行为规范
- Prompt Altitude（提示高度）应该处于合适的抽象层级
- 使用 XML 标签或 Markdown 清晰划定区块（如 `<background_information>`，`<instructions>`）
- 追求最小完备信息集（Minimal set of information），而不是单纯的“短”。

禁止：

- 硬编码：if else形式
- 过度抽象：未指导具体行为规范

示例：



（2） Few-shot设计：

设计原则：

- 质量大于数量
- 示例应该是多样化、规范化的典型示例 (Canonical examples)。

禁止：

- 把各种罕见的 Edge Cases（边缘情况）都塞进提示词里试图覆盖所有规则

（3）tool设计

设计原则

- **Token-efficient 促进效率：** 工具必须能够返回具有“Token 效率”的信息（避免冗长无效的信息，如数百行无用的 JSON 嵌套），并鼓励智能体表现出高效的行为。通过精简输入和输出，只保留推理必需的核心数据，从而保护大模型宝贵的上下文窗口不被冲刷。
- **Clear 目的明确：** 工具应该是自包含的（self-contained），具备极强的目的性。在微观设计上必须做到逻辑自洽、高内聚，确保该工具只专注于完成一件特定的事情，降低模型的认知门槛。
- **Non-overlapping 最小化功能重叠：** 确保各个工具之间的功能互不重叠。如果连人类工程师都无法明确在特定情况下该用哪个工具，AI 智能体同样无法做出正确判断。工具间的功能绝对隔离能消除语义歧义，有效解决模型在判断时产生的“路由错误”或随机选择倾向。
- **易于被 LLM 理解：** 工具的逻辑和定义必须契合大语言模型的理解方式。工具的命名、参数描述和返回结果需要像“摘要”一样精准，且发挥大模型固有的语言理解优势，绝不能产生模棱两可的理解。
- **Robust 具备鲁棒性：** 工具需要具备处理错误的能力，避免因单一工具调用失败（如 API 报错、超时）导致整个流程崩溃或陷入死循环。优秀的工具应能在内部捕获异常，并将代码错误转化为 LLM 能看懂的“自然语言纠错建议”，引导其自行修正重试。

管理方式：

不要给 Agent 塞一个臃肿庞大的工具集。如果连人类工程师都不知道某一步该用哪个工具，Agent 也会陷入选择困难并浪费上下文。

- **最小可行集合（Minimal Viable Set）**：精心策划并维持一个 Agent 所需的最少工具集合，确保上下文信息丰富且紧凑（informative, yet tight）

（4）记忆系统设计 (Memory Management)

记忆系统的结构设计

记忆系统通常设计为多层次的结构，模拟人类的记忆机制。这些层次包括短期记忆和长期记忆，分别用于处理当前任务的上下文和跨会话的知识。

- **短期记忆（Short-Term Memory）**：
  短期记忆主要用于存储当前对话的上下文，它的存储空间通常是有限的。在LLM中，这相当于输入的上下文窗口（Context Window）。短期记忆的作用是保持当前任务的连贯性，使得模型在短时间内能够理解并生成与当前对话相关的内容。
- **长期记忆（Long-Term Memory）**：
  长期记忆则存储那些跨多个对话会话、任务或用户交互的知识。它允许模型记住用户的历史记录、偏好设置、过去的决策等信息，以便在未来的对话中能够根据这些历史信息做出更加精确和

个性化的响应。长期记忆往往存储在外部数据库中，如向量数据库或知识图谱系统。

记忆管理

记忆管理不仅仅是将信息存储起来，它还涉及信息的有效更新、检索和删除。一个高效的记忆管理系统需要能够根据任务需求选择性地保存或忘记信息，以确保系统的性能和效率。

- **信息存储与更新**：
  在记忆系统中，信息是动态更新的。例如，短期记忆会根据新的对话内容不断被更新，而长期记忆则根据用户的历史信息不断积累。为了避免记忆过载，系统会通过优先级算法来决定哪些信息需要被保存，哪些可以被遗忘。这种信息选择与更新的过程确保了模型的效率和信息的相关性。
- **信息检索**：
  存储在记忆系统中的信息需要能够被快速检索出来。在长期记忆中，信息往往以向量的形式存储，模型可以通过检索算法（如基于语义相似度的向量检索）从记忆中召回最相关的信息，补充当前对话的上下文。这种信息检索不仅限于简单的关键词匹配，而是通过复杂的检索机制来选择最相关、最具信息量的内容。
- **信息压缩**：
  记忆系统需要处理大量的信息，而过多的信息会导致存储空间的浪费并影响性能。因此，压缩技术在记忆管理中扮演着重要角色。通过对存储的信息进行摘要、过滤或重组，系统能够将最重要的部分保留，并去除冗余信息。例如，模型可以通过生成一个摘要来替代冗长的对话记录，或只保留与当前任务高度相关的部分信息。

（5）知识注入通道（RAG）：

检索增强生成（Retrieval-Augmented Generation，RAG）是 **上下文工程中最核心的架构之一**。它通过将 LLM 与外部知识库、向量搜索和结构化检索系统结合，使生成结果能实时接入外部信息来源，从而解决 LLM 内部参数记忆的时效性和覆盖范围限制。

主要分为两类

模块化RAG（Modular RAG）

模块化RAG的核心思想是将整个RAG过程分解为多个独立的、可插拔的模块。这些模块分别负责检索、文档增强、生成等不同任务，每个模块可以根据需求进行独立的优化和替换。这种结构提供了极高的灵活性，使得系统能够针对不同的任务需求进行定制化设计。

在模块化RAG中，每个模块的功能明确且独立。

- **检索模块**负责从外部知识库中获取相关信息
- **查询重写模块**则优化查询以提高检索的相关性

- **文档增强模块**将检索到的信息整合并格式化
- **生成模块**利用增强后的信息生成最终答案。这样的设计使得模块化RAG能够灵活应对不同任务的需求，并且在不影响其他模块的情况下，优化单一模块的性能。

优势

- 灵活性与可扩展性：通过将不同的RAG流程拆分成模块，可以轻松替换或改进某一部分的实现，而不影响其他部分。
- 任务特定优化：针对特定任务，可以定制化每个模块。例如，对于某些任务可以采用更强的检索模块或改进查询重写算法，以适应不同的需求。
- 并行处理与优化：可以在模块化架构下并行处理不同的任务，如并行检索和生成过程，提升效率。

挑战

- **模块间协同问题**：由于每个模块独立执行，如何保证模块之间有效协同，避免信息丢失或冗余，仍然是一个挑战。
- **复杂性管理**：随着模块数量的增加，系统的整体管理和协调变得更加复杂，尤其是在多任务、多代理场景下。

代理式RAG（Agentic RAG）

代理式RAG 引入了 **智能体（Agent）** 的概念，突破了传统RAG的静态执行流程，将任务拆分为多个子任务，由不同的智能体（agent）来执行。这种方法使得RAG不仅仅是检索和生成的组合，更像是一个 **决策系统**，它能够在多个步骤中做出智能决策、动态调整策略，并在需要时调用外部工具进行进一步的操作。

代理式RAG通常包括以下几个组件：

- **智能体（Agent）**：agent是能够执行任务的实体，每个智能体都有一个明确的职责，如信息检索、任务分解、答案生成等。智能体能够进行 **自我规划**、**反思**、**迭代推理**，并根据当前的任务需求动态选择执行的步骤。
- **任务分解与规划模块（Task Decomposition and Planning Module）**：该模块负责将复杂的任务分解成多个子任务，然后根据每个子任务的性质选择合适的代理来处理。例如，对于一个包含多个问题的查询，任务规划模块会将问题拆解并分配给不同的代理，进行并行或依次处理。
- **工具调用模块（Tool-Calling Module）**：代理式RAG调用外部工具，如数据库查询、API接口、甚至是外部浏览器。在执行任务时，动态调用外部信息源进行补充。这种能力使得RAG更加适应动态且复杂的场景，特别是在需要从多个信息源获取数据的情况下。
- **反馈与反思模块（Feedback and Reflection Module）**：代理在任务执行过程中会不断根据反馈进行自我调整，尤其是在回答过程中，模型可能会发现某些部分的答案不准确或缺失，于是会

启动反思机制，重新检索或重新生成。这种自我调整能力类似于人类在问题解决过程中通过反思和反馈不断优化决策。

优势

- **动态决策能力**：通过代理智能体，RAG不仅执行任务，还能够**规划**和**自我调整**，使得系统能适应更复杂的情境。
- **工具集成能力**：代理能够调用外部工具进行补充和验证，使得模型不仅限于训练期间的知识，而是可以实时获取新的数据。
- **多步推理**：代理式RAG允许模型进行多轮推理和决策，在多阶段任务中能够根据每个阶段的输出动态调整后续的处理步骤。

挑战

- **智能体的协同与沟通**：多个智能体之间如何有效沟通、共享信息，避免重复工作或信息丢失，是代理式RAG面临的一大挑战
- **决策的效率与成本**：虽然代理能够进行决策和反思，但过于复杂的决策过程可能会导致系统效率低下，需要在灵活性和实时性之间找到平衡

动态上下文管理策略 (Dynamic Context Strategies)

很多人做 LLM 应用时，默认思路是："模型上下文不够，就想办法塞更多内容进去。"

但真正做过复杂系统后会发现，问题往往不是**塞得不够多**，而是**塞得不够准**。模型的上下文窗口、本次调用的 token 成本、注意力资源、推理延迟，这些都是真实存在的工程约束。上下文越长，不代表效果越好。信息一旦太多，模型反而更容易抓不住重点，甚至把真正重要的内容淹没在无关细节里。所以，动态上下文管理的核心不是"扩容"，而是**调度**：在每一次调用模型之前，系统都要判断——**这一次到底该给模型看什么，不该看什么，哪些要保留原文，哪些要压缩，哪些应该暂时放在外部，需要时再取回来。**

(1) 分层上下文

多 Agent 最合理的默认结构不是"大家共用一个大脑"，而是不同 agent 之间有合理的分层。 主 Agent 持有的是高价值、低噪声的控制层上下文：用户目标、成功标准、边界条件、当前计划、已确认结论、依赖关系和最终产物的位置。子 Agent 持有的是任务层上下文：某个子问题的任务说明、允许使用的工具、局部检索结果、临时工作笔记和中间探索轨迹。主 Agent 负责想清楚"做什么"，子 Agent 负责想清楚"怎么把这件事做完"。这正对应了 OpenAI 所说的 manager pattern：中央

manager 统一编排，其他 agent 作为工具型 worker 被调用；也对应 Anthropic 的 orchestrator-worker 结构：lead agent 做规划和综合，subagents 在独立窗口里并行做深入探索。

这也是为什么 plan-and-execute 在多智能体场景里很自然。高层规划 agent 不应该被长文、日志、搜索结果和失败重试污染；它的上下文应该尽量干净，专门保留任务目标、决策依据和阶段状态。执行型 agent 则相反，它天生就该处理“脏信息”：读取大文件、跑多轮搜索、做批量筛查、看长日志、解析表格、比对多个候选结果。LangChain 的 subagent 文档把这种模式讲得很直接：subagent 是 stateless 的，每次调用都在干净窗口里工作，这样可以避免主对话的 context bloat；Deep Agents 文档甚至把它叫作 context quarantine。

(2) 隔离与共享

所以，多 Agent 里的第一原则不是共享，而是先隔离，再选择性共享。

共享什么？ 共享那些会影响其他 agent 判断的“稳定事实”。 比如：任务目标、业务约束、输入数据版本、已完成步骤、关键结论、artifact 链接、依赖关系、下一步动作、失败原因摘要。

不共享什么？ 不共享那些只在局部探索中有价值的“过程噪声”。 比如：完整搜索日志、网页长摘录、原始工具返回、局部 scratchpad、试错路径、低置信度假设。

如果把后者也全量共享，系统很快就会退化成“所有人都在读彼此的施工现场”。而更成熟的做法，是让 agent 之间通过一个瘦身后的共享黑板协作：黑板上放状态和结论，不放整段原始过程。A2A 的设计理念也体现了这一点——agents 可以协同工作，但不需要互相暴露内部状态或内部逻辑。

(3) 标准化信息流转：任务移交与结论回流

跨 Agent 的信息传递，也最好不要理解成“把聊天上下文继续传下去”，而应该理解成移交一个任务包。 一个可落地的 handoff packet，至少应该包含这些内容：

- 当前要解决的子问题是什么；
- 这个子问题的边界和非目标是什么；
- 可用输入材料有哪些，哪些是附件，哪些只是参考；
- 输出应该长什么样，返回表格、摘要、候选列表还是结论；
- 什么时候停止，做到什么粒度就算完成；
- 结果如何引用，返回原文片段、链接还是 artifact ID。

因为 handoff 一旦只依赖自然对话历史，就很容易出现“指令漂移”——主 Agent 以为已经说清楚了，子 Agent 却抓错重点。Anthropic 在他们的多 agent 系统经验里专门强调，lead agent 必须把 objective、output format、tool guidance 和 task boundary 写清楚，否则 subagents 会重复工作、漏掉关键维度，甚至做完全不同的问题。

传结论上下文，不传原始对话日志。 很多人第一次做多 Agent，会天然觉得“既然前面都聊过了，那就把完整历史传给下一个 agent 吧”。但这通常不是继承上下文，而是继承噪声。OpenAI 的官方多 agent 文档已经把这件事说得很明确：主线程一旦被 exploration notes、test logs、stack traces、command output 之类的中间信息淹没，系统会出现 context pollution 和 context rot；因此更推荐

把 noisy work 移出主线程，让 sub-agents 返回 summaries，而不是原始中间输出。Anthropic 也同样把 subagents 的价值概括为 compression：它们在独立上下文里花更多 token 探索，再把高浓缩结论回流给 lead agent。

这就引出一个规范的返回格式设计：子 Agent 可以花数万 token 做探索，但最终只返回 1000–2000 token 的“高浓缩精华总结”。 这个总结最好不是一段松散自然语言，而是有固定结构。一个常见而稳定的格式是：

- 最终结论
- 支撑证据
- 置信度
- 仍未解决的问题
- 推荐下一步
- 产物引用

如果需要更强可组合性，还可以再加两个字段：“哪些内容已经验证过，不要重复做”；“哪些内容只是猜测，主 Agent 不要直接采用”。这样主 Agent 在汇总多个 worker 结果时，就不会把低质量猜测和高质量结论混在一起。关于 context engineering 的 Survey 中也提到了这个风险：多 agent 系统里，一个核心问题就是 inter-agent dependency opacity——不同 agent 可能在不一致的假设上工作，或者拿着冲突数据继续往前推；如果没有额外验证层和约束层，错误会被放大。

(4) 消息与产物的双层架构：基于引用的流转

从工程角度看，最稳妥的流转方式不是“消息传消息”，而是消息 + 产物引用。 也就是说，子 Agent 不一定把所有结果都塞回主 Agent 的 prompt。更好的做法常常是：

- 子 Agent 把长结果落到外部系统，比如文件、对象存储、数据库记录、工作区文档；
- 回给主 Agent 的只是一个短摘要，加上 artifact reference。

Anthropic 在公开总结里也提到过，某些结果可以绕过主协调器的长文本传输，直接写入文件系统，再把轻量引用返回给 coordinator，这样既减少“传话游戏”，又能降低上下文负担。

你可以把整个系统想成两层：消息层负责让 agent 之间传递意图、状态和简报；产物层负责保存长内容、原始证据和可复查材料。 **消息层求短，产物层求全。**

- 主 Agent 默认只看消息层，需要时再顺着引用去读取产物层。
- 这样才能同时兼顾“干净主线程”和“结果可追溯”。

(5) 全局状态控制面板

如果再往前走一步，多 Agent 系统其实需要一个明确的共享黑板模型。 但这里的“黑板”不要理解成所有内容都贴上去，而应该理解成一个只存全局稳定状态的共享面板。

它适合保存：

- 全局任务 ID 与子任务 ID
- 当前任务树和依赖关系
- 已完成 / 进行中 / 阻塞中的状态
- 关键决策和版本号
- 结论摘要
- artifact 指针
- 下一轮调度信息

它不适合保存：

- 全量推理轨迹
- 所有搜索结果
- 全部工具调用 raw output
- 大段中间日志
- 没有收敛的临时想法

黑板不是共享上下文的垃圾桶，而是共享状态的控制面板。一旦黑板开始堆原始材料，它就会反过来污染所有 agent。

(6) 防错机制与系统约束

当然，隔离带来的代价也很真实。如果上下文隔离做得太狠，系统会出现另一类问题：子 Agent 看不到必要背景，重复劳动变多，结果彼此冲突，或者主 Agent 失去对局部工作的真实理解。Anthropic 也公开提到，当前很多多 agent 系统仍然是同步执行的：lead agent 等一批 subagents 全部完成后再继续，这样协调简单，但信息流会形成瓶颈，主 Agent 不能实时 steer 子 Agent，子 Agent 之间也无法动态协同；如果改成异步，并行度会上去，但状态一致性、错误传播和结果协调又会变难。

所以，一个成熟的多 Agent 上下文流转系统，通常还要补上四个防错机制：

- **统一输出契约。** 不同子 Agent 必须按统一 schema 返回，不然主 Agent 很难可靠综合。
- **版本和引用管理。** 所有共享事实都要能追溯来源，至少知道它来自哪个子任务、哪一轮执行、对应哪个 artifact。
- **验证层。** 主 Agent 不应该无条件相信子 Agent 的摘要。重要结论最好再经过二次检查，或者交给 reviewer / verifier agent 复核。Survey 里也提到，多 agent 框架的一大风险就是过度依赖 LLM 自验证，缺少独立校验和补偿机制。
- **预算控制。** 并行 agent 很容易把 token 烧得很快。Anthropic 公开披露过，多 agent 系统虽然能显著提升某些复杂研究任务的表现，但 token 成本也会明显高于单 agent；因此它更适合高价值、可并行、信息量超过单窗口上限的任务，而不是所有请求都默认开多 agent。

Progressive disclosure（渐进式披露）——一种优雅的上下文工程设计哲学

如果说前面的章节讨论的是上下文工程的组成部分：系统提示、记忆、检索、工具、状态、压缩、路由与长期任务治理，那么这一章讨论的就是一个更底层、也更具有统摄性的原则：**渐进式披露（Progressive Disclosure）**。

把它放在最后，并不是因为它只是众多技巧中的一个补充项，恰恰相反，是因为当你把上下文工程一路分析到最后，会发现许多看似分散的方法，最终都在指向同一个判断：**不要一次性把所有信息都塞给模型，而要只在合适的时机，向它暴露当下真正需要的信息。** Anthropic 在谈上下文工程时，把问题的核心表述为：在有限的注意力预算下，找到“最小但高信号”的 token 集合；而在谈 Agent Skills 时，又把 progressive disclosure 作为核心加载机制，强调仅在相关时才展开完整说明和更深层资源。两者放在一起看，几乎已经构成了同一思想在两个层面的表述：一个是原则层，一个是系统实现层。

（1）什么是渐进式披露

“渐进式披露”并不是 AI 时代才出现的概念。它来自更早的交互设计领域。Jakob Nielsen 对它的经典定义是：**把高级的、低频的、次级的功能延后到第二层界面中，只先展示最重要、最常用的选项。**这样做的目的，是让系统更容易学习、更高效，也更不容易出错。NN/g 还特别指出，渐进式披露的价值不只是“隐藏复杂性”，更是在向用户传达一种优先级：**凡是默认出现在第一屏的信息，都是当前最重要的信息。**

如果把这个定义直接迁移到 LLM 系统里，意思就变成了：**不要让模型在第一时间面对一个臃肿、混杂、平铺直叙的上下文大包，而是要让它先看到最关键的线索，再根据任务推进逐层展开更深的内容。** 从“先展示主要按钮，再把高级设置藏到二级菜单里”，到“先给模型元数据，再按需加载详细说明和文件”，其结构是完全同构的。不同的只是交互对象从“人类用户”变成了“作为推理主体的模型”。

（2）为什么它会成为上下文工程的核心原则

上下文工程之所以重要，不是因为今天的模型上下文窗口还不够大，而是因为**上下文本身从来都不是越多越好，而是一种稀缺的注意力资源分配问题**。Anthropic 在《Effective context engineering for AI agents》中强调，LLM 受限于有限的 attention budget，好的上下文工程意味着找到“最小可能的高信号 token 集”，从而最大化获得目标结果的概率。文章同时指出，随着上下文长度增加，模型并不是突然失效，而是会出现精度下降、长距离依赖变弱、相关性判断变差等渐进式退化现象；因此，上下文必须被看作一种**边际收益递减**的有限资源。

这意味着，一个成熟的 Agent 系统真正要解决的，不是“如何拥有更多上下文”，而是“如何避免让无关上下文污染当前决策”。也正是在这个意义上，渐进式披露不再只是一个 UI 设计技巧，而变成了上下文工程的根本方法论：**把上下文视为一系列可分层、可延迟、可条件触发的信息单元，而不是一次性整体注入的静态文本。** 当前只暴露必要线索；当模型的下一步行动证明某部分信息确实相关时，再把那一层展开；如果仍不够，再继续向更细的文件、脚本、数据、状态读取。这就是从“prompt thinking”走向“context architecture”的关键一步。

（3）渐进式披露不是上下文工程的“旁支”，而是它的落地形式

很多人在第一次听到这两个词时，会把它们当成并列关系：上下文工程是一回事，渐进式披露是另一回事。更准确的理解应该是：**上下文工程是总问题，渐进式披露是这个总问题的一种核心解法。**

上下文工程关心的是：什么信息该进入模型、何时进入、以什么形式进入、如何在长任务中保留与裁剪。它研究的是上下文的整体组织。渐进式披露则给出了其中一个极其关键的答案：**不要预先决定所有信息都该进入，而要让系统在运行过程中逐层揭示。** Anthropic 对 agentic search 的描述很清楚：越来越多团队正在从“预先检索完再一次性喂给模型”，转向“保留轻量标识符，如文件路径、查询、链接等，然后在运行时按需把数据加载进上下文”的 just-in-time 策略；而让 agent 自主导航和检索，又进一步“enables progressive disclosure”，也就是通过探索逐步发现相关上下文。

所以，从概念层级上说，**上下文工程更像一门调度学，渐进式披露更像它的第一性原则之一**。前者问的是“如何设计上下文系统”，后者回答的是“设计时应默认采用怎样的信息暴露哲学”。如果前文你的文章已经依次讨论过 RAG、记忆、状态、工具、长上下文压缩等主题，那么把渐进式披露放在最后会非常自然，因为它能把前面的技术线全部收束起来：这些方法表面不同，本质上都在试图做到同一件事——**只在必要的时候，把必要的信息，交给模型。**

（4）它和 Just-in-Time 是什么关系

渐进式披露经常和 just-in-time 一起出现，但两者并不完全相同。可以把它们理解为两个互补维度。

Just-in-time 更偏时间维度。 它关心的是：信息什么时候取进来。不是预先把“所有可能有用”的内容都塞进上下文，而是在推理进行到某一步时，才根据当前需要去检索、读取、执行。Anthropic 的表述就是：agent 维护的是轻量级引用，而不是完整数据本体；通过工具在运行时动态加载内容。

Progressive disclosure 更偏结构维度。 它关心的是：信息按几层来揭示。先给目录级线索，再给概要，再给细节，再给脚本或原始数据。它不是简单地“晚一点给”，而是要有明确的层级设计，让系统知道第一层应该只承载方向感，第二层承载任务说明，第三层才承载执行细节。Anthropic 在 Skills 架构里将这种思想制度化为三层：元数据先加载；完整 `SKILL.md` 只在相关时加载；更深层链接文件与资源只在需要时访问。

因此，可以把两者的关系概括为：**JIT 解决“何时加载”，Progressive Disclosure 解决“按什么层级加载”。** 在一个优秀的 agent 系统中，它们往往同时成立。系统不会一次性预装全部内容，这是 JIT；系统也不会在第一次命中时就把所有相关文件全读进来，而是先读最上层说明，再按引用深入，这是 Progressive Disclosure。两者结合，才真正构成现代上下文工程里的“按需上下文化”能力。

（5）Agent Skills 为什么是这个原则最清晰的产品化实现

如果要找一个最能说明渐进式披露的具体系统，Anthropic 的 Agent Skills 几乎是现成的教材。Anthropic 在官方文档中把 Skill 定义为一种模块化能力包，里面可以包含指令、元数据，以及可选的脚本、模板和资源。Skill 的关键不在于它只是一个“提示词文件夹”，而在于它把**知识、流程、代码和参考材料**组织成一个可以被 Agent 按需发现和读取的文件系统对象。

更重要的是，Skills 的发现与加载本身就是渐进式披露。模型在启动时并不会把所有 Skill 的全部内容都装进上下文；它先只看到 `name` 和 `description` 这样的元数据，用来知道“有哪些能力存在”；只有当某个 Skill 与当前任务相关时，才进一步读取 `SKILL.md` 的完整说明；而如果 `SKILL.md` 又链接到更详细的参考文档、样例或脚本，模型也只会在需要时继续打开它们。Cookbook 和官方文档都强调，这种架构的好处在于“只为真正使用的内容付费”，未被访问的内容不占用上下文。

这件事的意义非常大。它表明上下文工程已经不再只是“怎么写 prompt”，而是发展成了**怎么组织能力资产**。以前我们把专业知识灌进系统提示；现在更合理的做法是把它们整理成分层文档、样例、脚本和资源，让模型在真正需要时去发现、调用和执行。也就是说，**Skill 并不是对上下文工程的替代，而是渐进式披露在工程系统中的制度化形态。**

（6）为什么这比“把所有相关知识都提前放进去”更优

直觉上，很多人会认为“我多给点背景，模型总归更懂”。这在简单问答里有时成立，但在复杂 Agent 场景里，常常会变成灾难。因为上下文一旦变长，问题就不再只是“够不够”，而是“有没有把当前决策真正需要的信号淹没”。Anthropic 明确提醒，长上下文会带来 context pollution 和 relevance concerns，即便未来上下文窗口继续扩大，这类问题依然存在。

渐进式披露优于“全集合预装”的原因，至少有四个。

- 它降低了认知竞争。

模型不需要在一开始就同时权衡几十份说明、上百条规范、多个看似相关的工具接口，而是只处理当前层次最关键的线索。这样做并不是让模型“知道得更少”，而是让它**在当前时刻更清楚什么最重要**。这与 NN/g 对人类用户的分析高度一致：隐藏次级选项能够提升可学性、效率并减少错误率。

- 它更接近真实世界中的专业工作方式。

人类专家并不是把全部知识同时压在工作记忆里，而是依赖外部组织结构：目录、标签、书签、索引、模板、速查表。Anthropic 也用类似的比喻说明，模型通过轻量引用在运行时取回信息，正如人不会记住整个语料库，而是依靠文件系统、邮箱、书签等外部结构按需检索。

- 它允许系统把“信息”与“行动”解耦。

很多时候模型并不需要把一整段脚本源码读进上下文，它只需要知道有这样一个脚本可供执行；很多时候它也不需要先读完整数据库，只要能写出针对性查询并查看结果切片即可。Anthropic 在 code execution with MCP 中也强调，按需加载工具、在信息进入模型前先过滤数据、把复杂逻辑压缩到一次执行里，都能让 Agent 更高效地使用上下文。

- 它天然适合组合式系统。

一个现实 Agent 往往不是只面对一种知识源，而是同时涉及规范、品牌规则、历史记忆、文件库、API、工作流、工具和用户状态。如果所有内容都必须预先装入上下文，那么系统规模几乎无法增长；而如果它们都能采用渐进式披露，系统就可以在保持可扩展性的同时，维持较干净的工作记忆。

（7）把渐进式披露落实到上下文工程中，应该如何设计

如果把它写成一个工程原则，可以表述为：

默认不注入；只给索引。
先向模型提供结构化的方向感，而不是全部内容本体。比如只暴露文件名、路径、更新时间、标题、摘要、标签、Skill 描述、工具 schema，而不先塞完整正文。Anthropic 特别提到，文件层级、命名约定、时间戳本身就是重要信号，它们能帮助 agent 判断下一步应读取什么。

先给概览，再给细则。
`SKILL.md` 应该像 onboarding guide 的目录页，提供高层路线而非所有细节。Anthropic 的 best practices 也明确建议把 `SKILL.md` 视为 overview，像目录一样指向更详细材料，并在内容接近上限时把内容拆到独立文件中。

能执行就别展开。
如果某段逻辑已经被封装为稳定脚本、查询模板或工具调用，优先让 Agent 直接执行，而不是把底层实现全部灌进提示。这样做既节省 token，也降低了模型在“读懂实现细节”上的注意力消耗。Skills Cookbook 反复强调，skills 的价值之一就在于可以携带经过验证的 helper scripts，减少从头生成代码的成本和错误率。

把探索设计成一条有反馈的路径。
Anthropic 对 agentic exploration 的描述很重要：每一次交互都会产出新的上下文，进而影响下一步决策。文件大小暗示复杂度，命名暗示用途，时间戳暗示新旧，目录结构暗示角色。这意味着渐进式披露不是静态分页，而是**边探索、边判断、边缩小范围**的递归过程。

（8）它的代价与边界：为什么不是所有场景都应完全按需

把渐进式披露说成“终局原则”，并不意味着一切场景都应该走到极端。Anthropic 同样提醒，运行时探索的代价是速度更慢，而且如果没有合适的工具和启发式规则，Agent 可能会误用工具、走入死路，甚至浪费更多上下文。

因此，成熟系统通常采用的是**混合策略**。也就是：先把少量高价值、几乎必定相关的内容直接放进上下文，以保证起步速度；再允许 Agent 在后续步骤中自主探索其他材料。Anthropic 明确举例说，Claude Code 就采用这种混合模式：`CLAUDE.md` 会被直接放入上下文，而 `glob`、`grep` 等原语则支持它在环境中继续 just-in-time 地读取文件。对于内容较稳定、结构较清晰的领域，如法律或金融，这种“少量预装 + 后续探索”的混合方案可能尤其合适。

这也提醒我们：渐进式披露不是教条式地“越晚给越好”，而是要求我们判断**哪一层信息具有足够高的先验相关性，值得预装；哪一层则应该等证据出现后再展开。** 换句话说，它不是反对上下文注入，而是反对**未经分层和优先级设计的粗暴注入**。这个分寸感，恰恰是上下文工程最难也最重要的部分。

上下文工程真正成熟的标志，不是我们终于找到了一个能容纳百万 token 的大窗口，也不是我们能把越来越多的知识和工具接到模型上，而是我们开始接受一个更克制的现实：**能力不来自信息的堆积，而来自信息暴露方式的设计。** Anthropic 在上下文工程文章最后重申，无论是做长任务压缩、设计高效工具，还是让 Agent 在环境中 just-in-time 地探索，核心原则始终是寻找“最小的高信号 token 集”。如果把这句话翻译成系统设计语言，它就是渐进式披露。